

Computational perspectives on the classical decision problem

Lecture 2 (Prague 10.08.2026)

Bartosz Jan Bednarczyk `bartek@cs.uni.wroc.pl`

TECHNISCHE UNIVERSITÄT WIEN & UNIwersYTET WROCLAWSKI



TECHNISCHE
UNIVERSITÄT
WIEN
Vienna | Austria



Uniwersytet
Wrocławski

Agenda

Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

3. **NEXPTIME-easyness**: small models

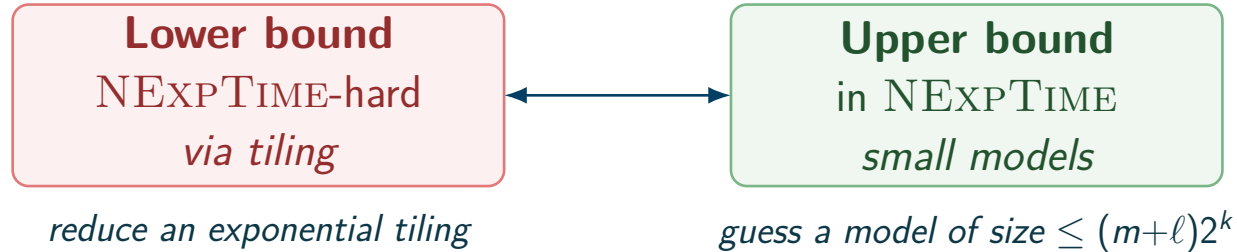
Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

3. NEXPTIME-**easyness**: small models



Agenda

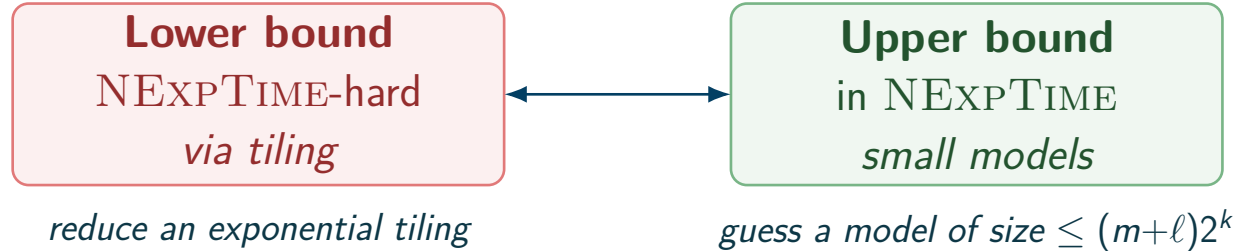
1. Complexity of **Monadic** Fragment of First-Order Logic.

MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

3. NEXPTIME-easyness: small models

- **Warm-up**: no constants, no equality.



Agenda

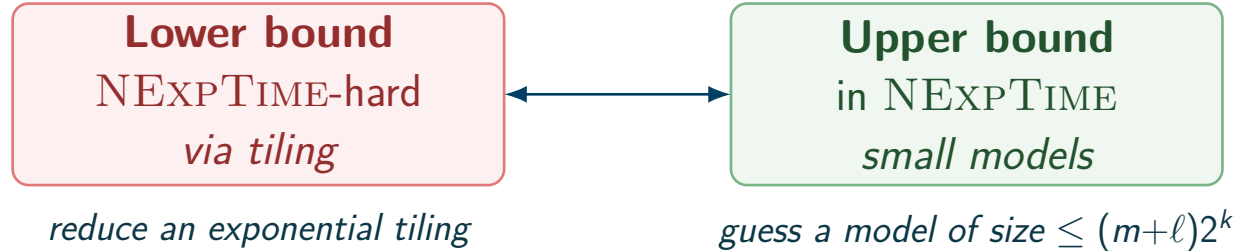
1. Complexity of **Monadic** Fragment of First-Order Logic.

MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

3. NEXPTIME-easyness: small models

- **Warm-up**: no constants, no equality.
- Tackling the **general case**.



Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

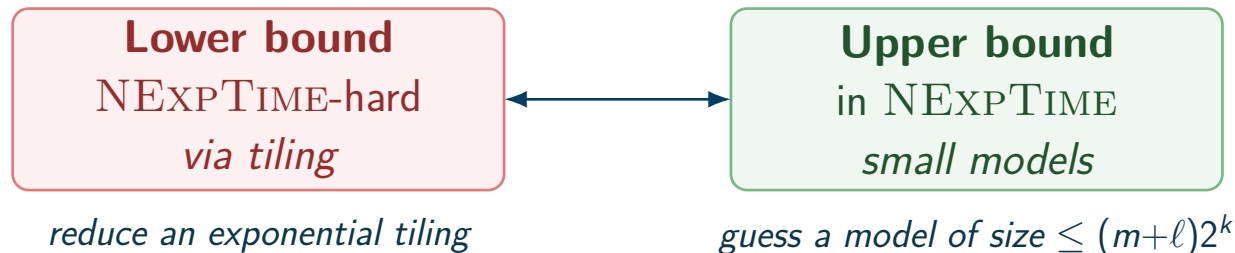
MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

3. NEXPTIME-easyness: small models

- **Warm-up**: no constants, no equality.
- Tackling the **general case**.

4. NEXPTIME-hardness: via tiling



Agenda

1. Complexity of **Monadic** Fragment of First-Order Logic.

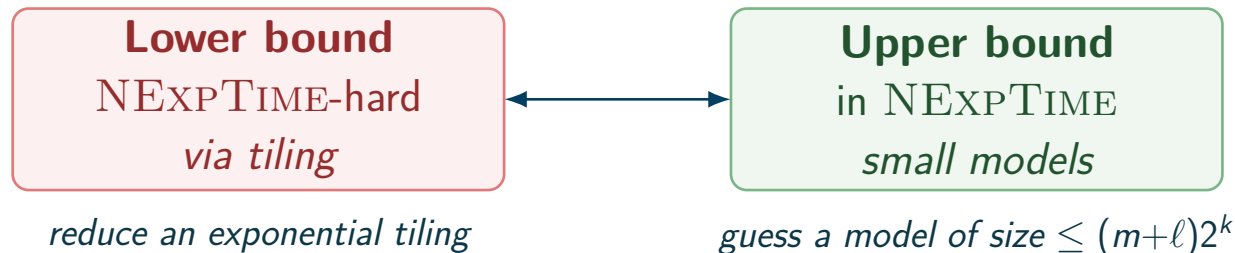
MFO = FO + only **unary** predicates + **optionally** $\approx$ and constants

2. **One-types**: the combinatorial object driving both lower and upper bounds.

3. NEXPTIME-easiness: small models

- **Warm-up**: no constants, no equality.
- Tackling the **general case**.

4. NEXPTIME-hardness: via tiling



Feel free to ask questions and interrupt me!

Please don't be shy! If needed, send me an email (bartek@cs.uni.wroc.pl) or approach me before/after the lecture!

If you would like to do a research project on related topics, let me know!

Icon downloaded from icon-icons.com by ©VisualPharm (under CC BY 4.0). No changes have been made.

An Introduction to the Monadic Fragment of First-Order Logic

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”
2. $\exists x (R(x) \wedge \neg S(x))$ “something is R but not S ”



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”
2. $\exists x (R(x) \wedge \neg S(x))$ “something is R but not S ”
3. $\forall x \neg (P(x) \wedge Q(x))$ “ P and Q never coincide”



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”
2. $\exists x (R(x) \wedge \neg S(x))$ “something is R but not S ”
3. $\forall x \neg (P(x) \wedge Q(x))$ “ P and Q never coincide”
4. $\exists x P(x) \wedge \exists x \neg P(x)$ “some are P , some are not”



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”
2. $\exists x (R(x) \wedge \neg S(x))$ “something is R but not S ”
3. $\forall x \neg (P(x) \wedge Q(x))$ “ P and Q never coincide”
4. $\exists x P(x) \wedge \exists x \neg P(x)$ “some are P , some are not”
5. $P(a)$ “the constant a satisfies P ”



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”
2. $\exists x (R(x) \wedge \neg S(x))$ “something is R but not S ”
3. $\forall x \neg (P(x) \wedge Q(x))$ “ P and Q never coincide”
4. $\exists x P(x) \wedge \exists x \neg P(x)$ “some are P , some are not”
5. $P(a)$ “the constant a satisfies P ”
6. $\exists x \exists y \exists z x \not\approx y \wedge y \not\approx z \wedge z \not\approx x \wedge P(x) \wedge P(y) \wedge P(z)$
“at least three distinct elements satisfy P ”



Löwenheim (1915): MFO is decidable.

An Introduction to the Monadic Fragment of First-Order Logic

Definition (Monadic Fragment of First-Order Logic (MFO))

- Only unary (*monadic*) predicates $P_1, \dots, P_k$;
- No function symbols;
- Optionally constants $c_1, \dots, c_m$ and equality $\approx$.

Examples of MFO formulae.

1. $\forall x (P(x) \rightarrow Q(x))$ “every P is a Q ”
2. $\exists x (R(x) \wedge \neg S(x))$ “something is R but not S ”
3. $\forall x \neg (P(x) \wedge Q(x))$ “ P and Q never coincide”
4. $\exists x P(x) \wedge \exists x \neg P(x)$ “some are P , some are not”
5. $P(a)$ “the constant a satisfies P ”
6. $\exists x \exists y \exists z x \not\approx y \wedge y \not\approx z \wedge z \not\approx x \wedge P(x) \wedge P(y) \wedge P(z)$
“at least three distinct elements satisfy P ”



Löwenheim (1915): MFO is decidable.



Cannot express properties of binary relations.

Unary Types: The main tool for today's lecture

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Palette



Colours



Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Palette

Colours

Example

Suppose $\sigma :=$  P_1  P_2  P_3

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Palette

Colours

Example

Suppose $\sigma :=$



P_1



P_2



P_3



$\text{tp} = \{P_1, P_2\}$



$\text{tp} = \{P_2\}$



$\text{tp} = \{P_1, P_3\}$



$\text{tp} = \emptyset$



$\text{tp} = \{P_1, P_2\}$

...

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Palette

Colours

Example

Suppose $\sigma :=$



P_1



P_2



P_3



$\text{tp} = \{P_1, P_2\}$



$\text{tp} = \{P_2\}$



$\text{tp} = \{P_1, P_3\}$



$\text{tp} = \emptyset$



$\text{tp} = \{P_1, P_2\}$

...



Nom. of 1-types?

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Palette

Colours

Example

Suppose $\sigma :=$



P_1



P_2



P_3



$\text{tp} = \{P_1, P_2\}$



$\text{tp} = \{P_2\}$



$\text{tp} = \{P_1, P_3\}$



$\text{tp} = \emptyset$



$\text{tp} = \{P_1, P_2\}$

...



Nom. of 1-types?
 $2^{|\sigma|}$

Unary Types: The main tool for today's lecture

Let $\sigma := \{P_1, \dots, P_k\}$ consists of k unary predicates, and a be an element of a σ -structure $\mathfrak{A}$.

Definition (1-type of a in $\mathfrak{A}$) $\text{tp}^{\mathfrak{A}}(a) := \{P \in \sigma \mid \mathfrak{A} \models P(a)\}.$

Palette

Colours

Example

Suppose $\sigma :=$



P_1



P_2



P_3



$\text{tp} = \{P_1, P_2\}$



$\text{tp} = \{P_2\}$



$\text{tp} = \{P_1, P_3\}$



$\text{tp} = \emptyset$



$\text{tp} = \{P_1, P_2\}$

...



Nom. of 1-types?
 $2^{|\sigma|}$

Good to remember: 1-types are complete descriptions of elements.

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO.

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

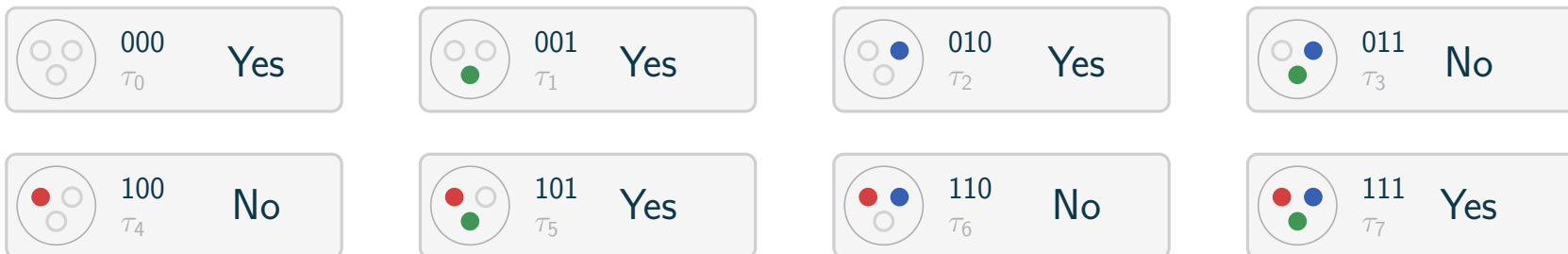
1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$?

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$? Only the existence of types matters!

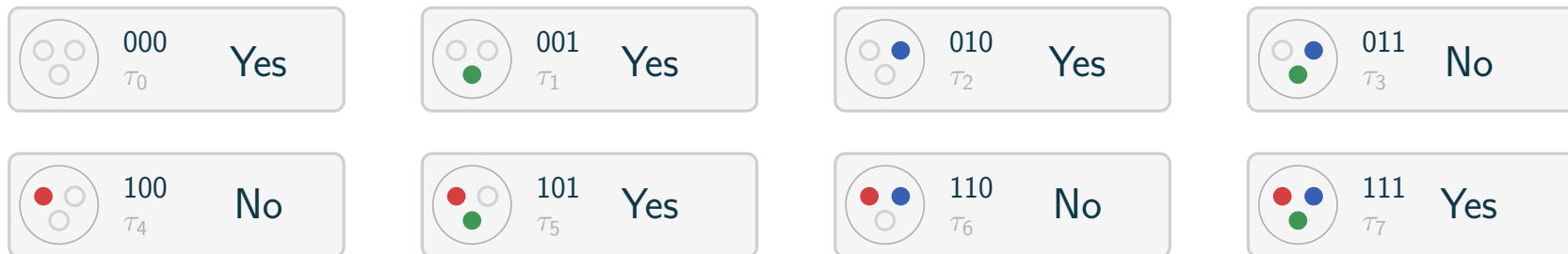
Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$? Only the existence of types matters!



Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$? Only the existence of types matters!



3. Question: How to get a small model?

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

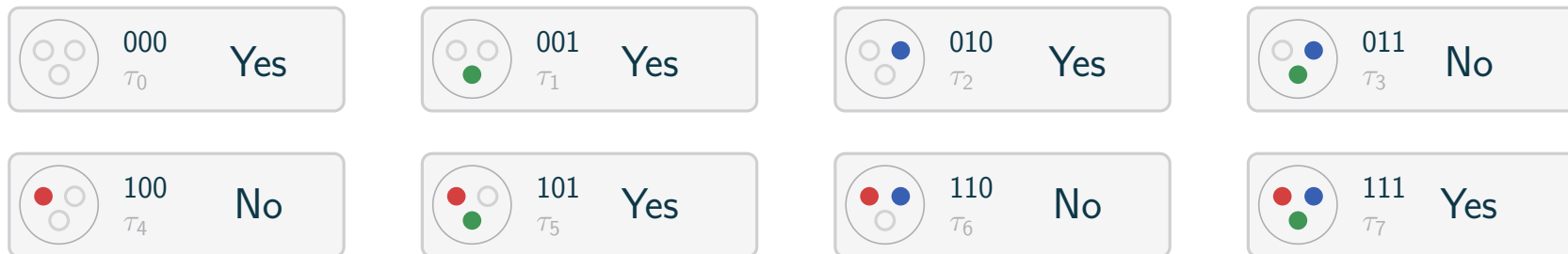
1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$? Only the existence of types matters!



3. Question: How to get a small model? Collapse indistinguishable elements!

Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$? Only the **existence of types** matters!



3. Question: How to get a **small** model? **Collapse indistinguishable** elements!

$$a \sim b \text{ iff } \text{tp}^{\mathfrak{A}}(a) = \text{tp}^{\mathfrak{A}}(b)$$

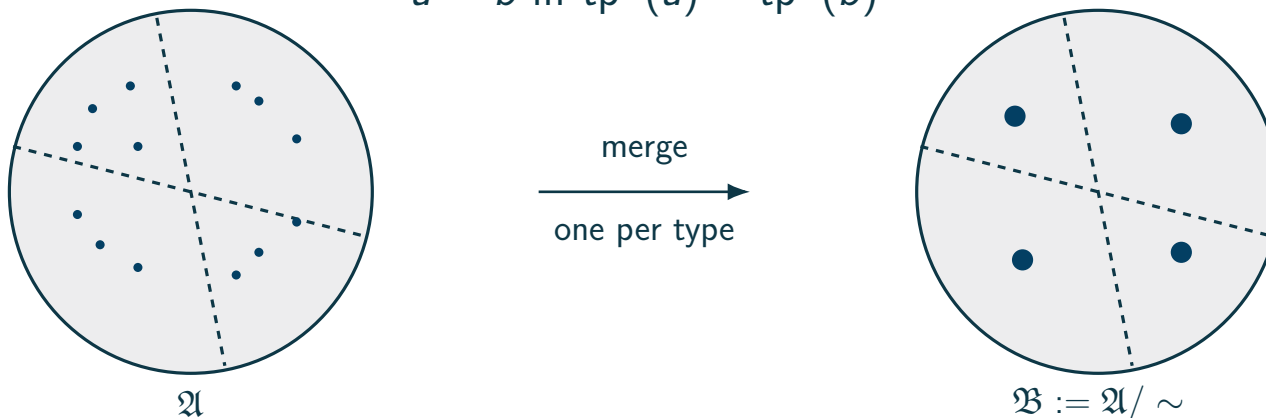
Warm-up: Solving SAT for Pure MFO (no constants and $\approx$)

1. Take an input formula φ in MFO. Is there $\mathfrak{A} \models \varphi$?
2. Question: How does φ view an intended model $\mathfrak{A}$? Only the existence of types matters!



3. Question: How to get a small model? Collapse indistinguishable elements!

$$a \sim b \text{ iff } \text{tp}^{\mathfrak{A}}(a) = \text{tp}^{\mathfrak{A}}(b)$$

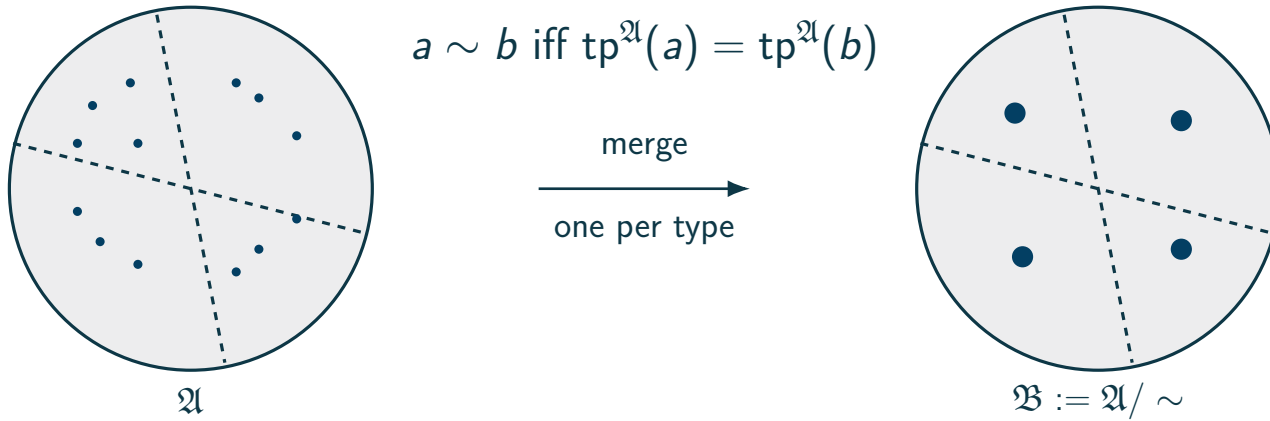


Warm-up: Solving SAT for Pure MFO. Part II: The proof.

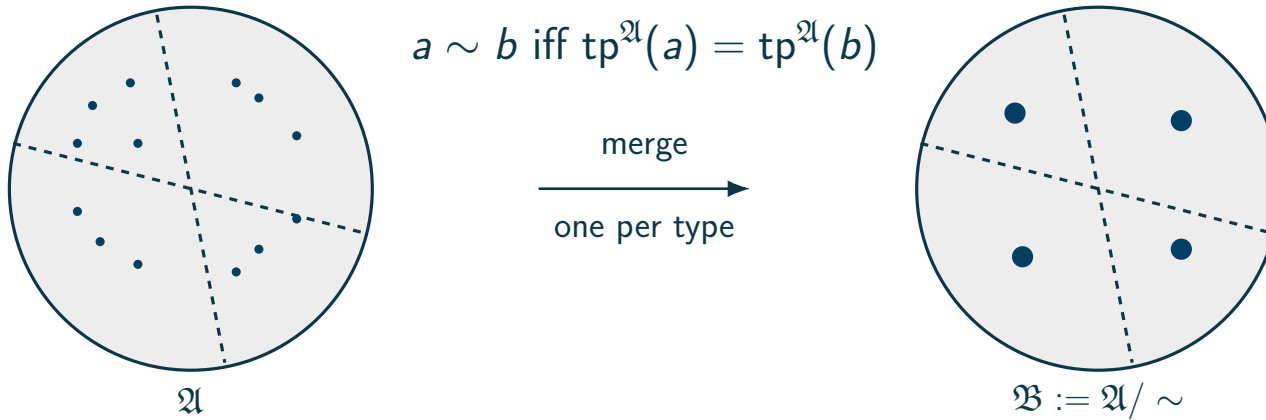
Warm-up: Solving SAT for Pure MFO. Part II: The proof.

$$a \sim b \text{ iff } \text{tp}^{\mathfrak{A}}(a) = \text{tp}^{\mathfrak{A}}(b)$$

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



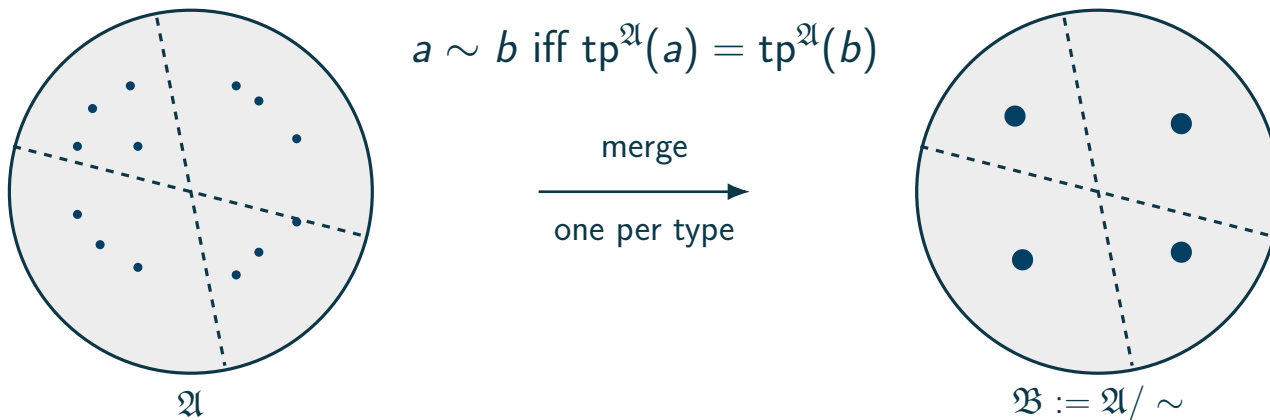
Warm-up: Solving SAT for Pure MFO. Part II: The proof.



Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Warm-up: Solving SAT for Pure MFO. Part II: The proof.

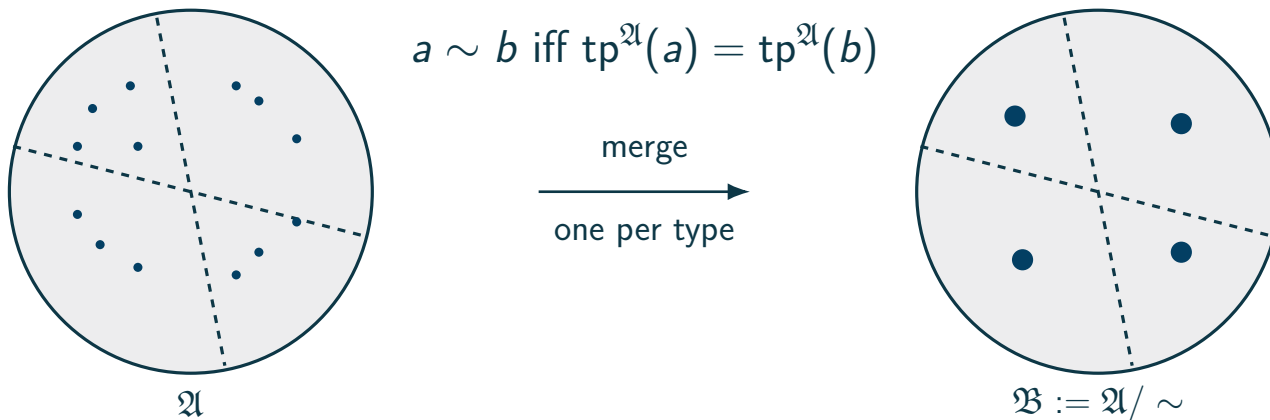


Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



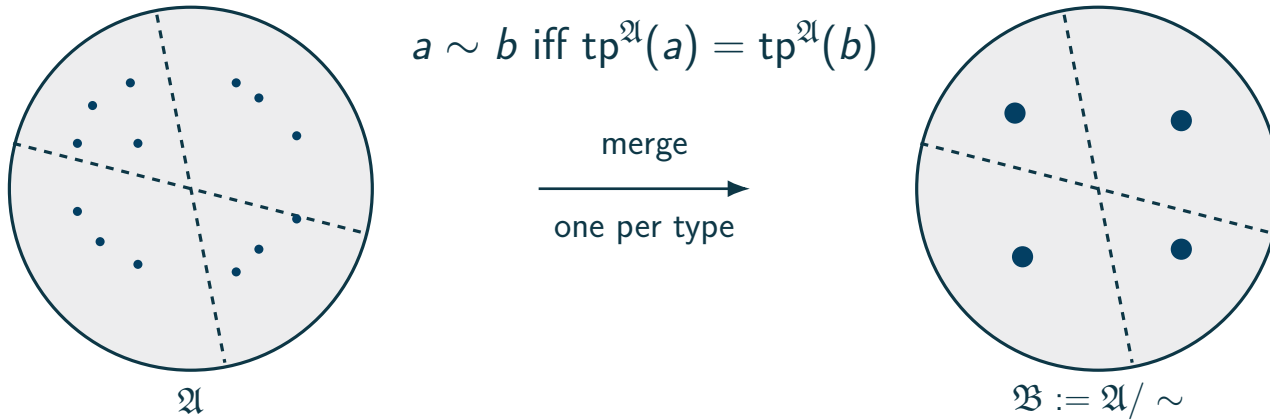
Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic.

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



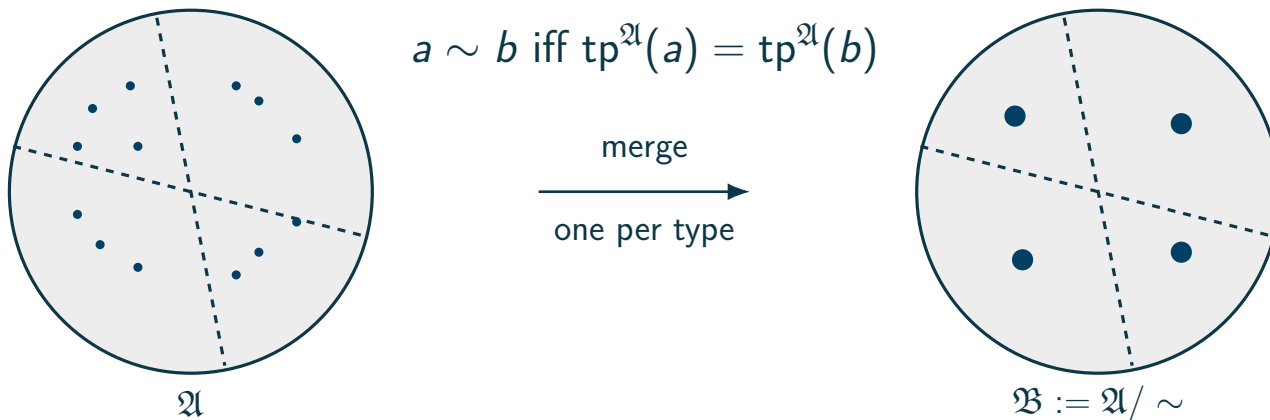
Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic. $\rightsquigarrow a \in A$ and $h(a) \in B$ have the same type (for all $a \in \bar{a}$).

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



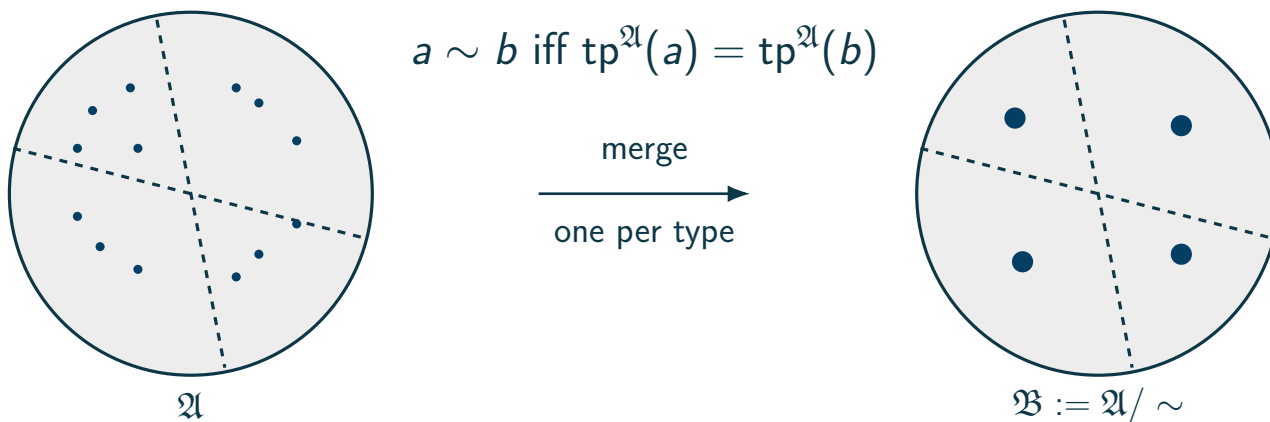
Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic. $\rightsquigarrow a \in A$ and $h(a) \in B$ have the same type (for all $a \in \bar{a}$).
2. ψ boolean combination.

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



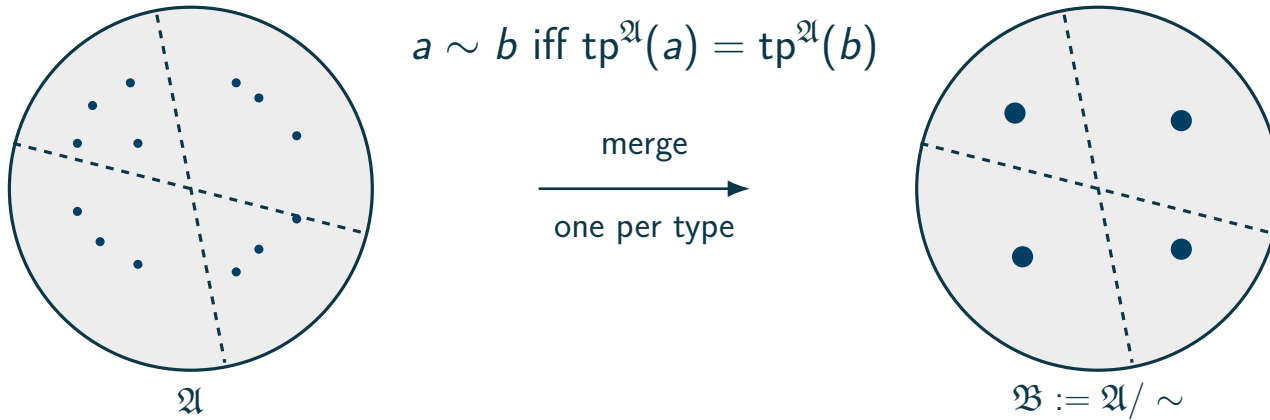
Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic. $\rightsquigarrow a \in A$ and $h(a) \in B$ have the same type (for all $a \in \bar{a}$).
2. ψ boolean combination. $\rightsquigarrow$ inductive hypothesis + properties of $\models$.

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



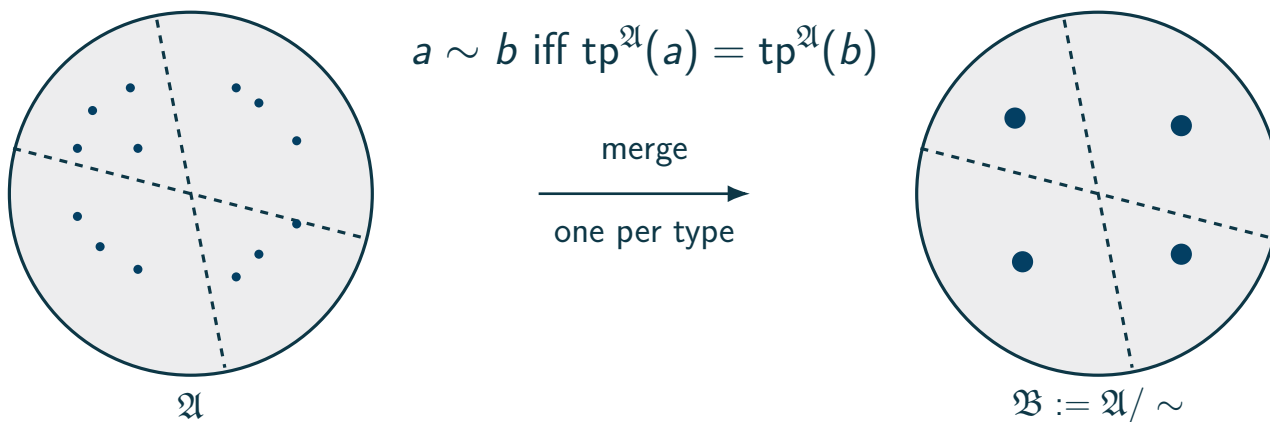
Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic. $\rightsquigarrow a \in A$ and $h(a) \in B$ have the same type (for all $a \in \bar{a}$).
2. ψ boolean combination. $\rightsquigarrow$ inductive hypothesis + properties of $\models$.
3. $\psi = \exists x \theta(x, \bar{y})$.

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



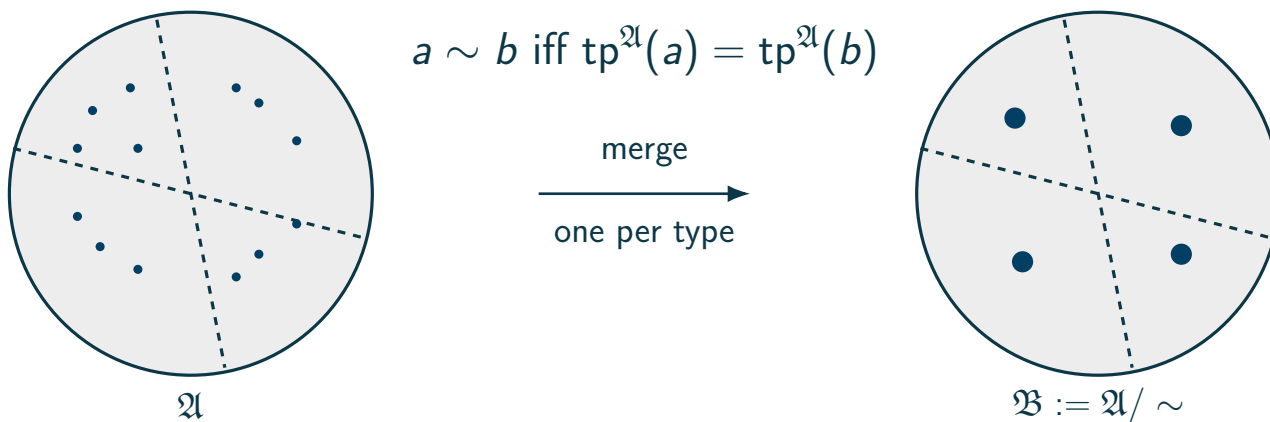
Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic. $\rightsquigarrow a \in A$ and $h(a) \in B$ have the same type (for all $a \in \bar{a}$).
2. ψ boolean combination. $\rightsquigarrow$ inductive hypothesis + properties of $\models$.
3. $\psi = \exists x \theta(x, \bar{y})$. $\rightsquigarrow$ a representative of each equivalence class survives + inductive hypothesis.

Warm-up: Solving SAT for Pure MFO. Part II: The proof.



Lemma

$|\mathfrak{B}| \leq 2^{|\sigma|} +$ For all $\psi \in \varphi$ and $\bar{a} \in A$ we have: $\mathfrak{A} \models \psi[\bar{a}]$ iff $\mathfrak{B} \models \psi[h(\bar{a})]$ for the quotient map h .

Proof (by induction on ψ).

1. ψ atomic. $\rightsquigarrow a \in A$ and $h(a) \in B$ have the same type (for all $a \in \bar{a}$).
2. ψ boolean combination. $\rightsquigarrow$ inductive hypothesis + properties of $\models$.
3. $\psi = \exists x \theta(x, \bar{y})$. $\rightsquigarrow$ a representative of each equivalence class survives + inductive hypothesis.

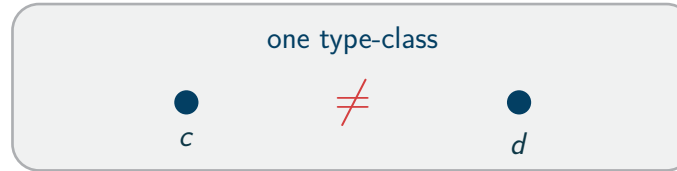
Theorem

Every satisfiable pure MFO sentence φ has a model of size $\leq 2^{|\varphi|}$. SAT(pure MFO) is in NEXPTIME.

Full MFO: why the previous approach breaks?

Full MFO: why the previous approach breaks?

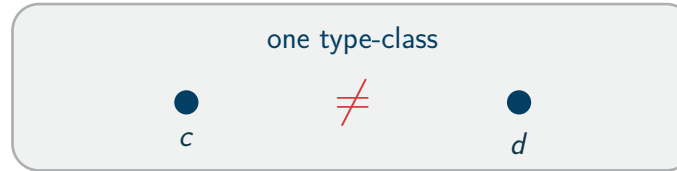
1. In the presence of constants or $\approx$ one representative per type may not be enough.



Full MFO: why the previous approach breaks?

1. In the presence of constants or $\approx$ one representative per type may not be enough.

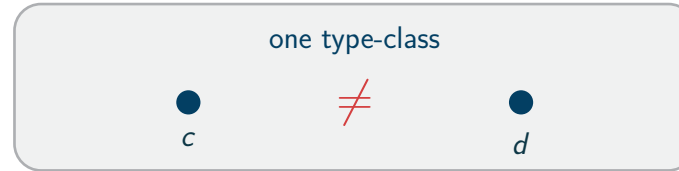
- Consider: $c \not\approx d \wedge \bigwedge_{P \in \sigma} P(c) \leftrightarrow P(d)$



Full MFO: why the previous approach breaks?

1. In the presence of constants or $\approx$ one representative per type may not be enough.

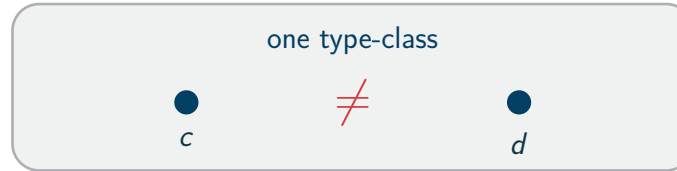
- Consider: $c \not\approx d \wedge \bigwedge_{P \in \sigma} P(c) \leftrightarrow P(d)$
- or: $\exists x \exists y x \not\approx y \wedge \bigwedge_{P \in \sigma} P(x) \leftrightarrow P(y)$



Full MFO: why the previous approach breaks?

1. In the presence of constants or $\approx$ one representative per type **may not be enough**.

- Consider: $c \not\approx d \wedge \bigwedge_{P \in \sigma} P(c) \leftrightarrow P(d)$
- or: $\exists x \exists y x \not\approx y \wedge \bigwedge_{P \in \sigma} P(x) \leftrightarrow P(y)$

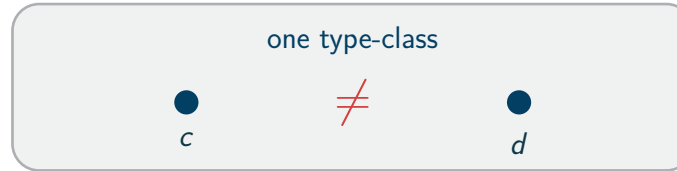


- A formula with ℓ quantifiers and m constants may enforce $\leq (m + \ell)$ representatives per 1-type.

Full MFO: why the previous approach breaks?

1. In the presence of constants or $\approx$ one representative per type **may not be enough**.

- Consider: $c \not\approx d \wedge \bigwedge_{P \in \sigma} P(c) \leftrightarrow P(d)$
- or: $\exists x \exists y x \not\approx y \wedge \bigwedge_{P \in \sigma} P(x) \leftrightarrow P(y)$



- A formula with ℓ quantifiers and m constants may enforce $\leq (m + \ell)$ representatives per 1-type.

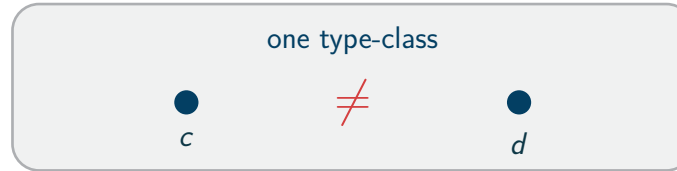
$$\exists x_1 \exists x_2 \dots \exists x_\ell \bigwedge_{i=1}^{\ell} \bigwedge_{j=1}^m \bigwedge_{i' \neq i, j \neq j'} x_i \not\approx x_{i'} \wedge x_i \not\approx c_j \wedge c_j \not\approx c_{j'} \wedge \bigwedge_{P \in \sigma} P(x_i) \leftrightarrow P(x_j) \wedge P(x_i) \leftrightarrow P(c_j)$$

Full MFO: why the previous approach breaks?

1. In the presence of constants or $\approx$ one representative per type **may not be enough**.

- Consider: $c \not\approx d \wedge \bigwedge_{P \in \sigma} P(c) \leftrightarrow P(d)$

- or: $\exists x \exists y x \not\approx y \wedge \bigwedge_{P \in \sigma} P(x) \leftrightarrow P(y)$



- A formula with ℓ quantifiers and m constants may enforce $\leq (m + \ell)$ representatives per 1-type.

$$\exists x_1 \exists x_2 \dots \exists x_\ell \bigwedge_{i=1}^{\ell} \bigwedge_{j=1}^m \bigwedge_{i' \neq i, j \neq j'} x_i \not\approx x_{i'} \wedge x_i \not\approx c_j \wedge c_j \not\approx c_{j'} \wedge \bigwedge_{P \in \sigma} P(x_i) \leftrightarrow P(x_j) \wedge P(x_i) \leftrightarrow P(c_j)$$

Lemma

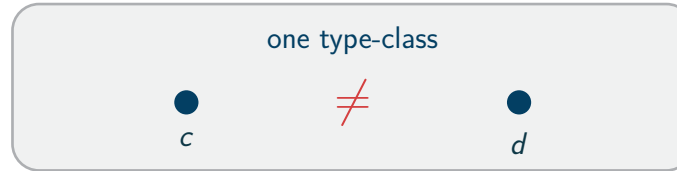
Satisfiable φ with k predicates, m constants, ℓ quantifiers has a model with $\leq (m + \ell) \cdot 2^k$ elements.

Full MFO: why the previous approach breaks?

1. In the presence of constants or $\approx$ one representative per type **may not be enough**.

- Consider: $c \not\approx d \wedge \bigwedge_{P \in \sigma} P(c) \leftrightarrow P(d)$

- or: $\exists x \exists y x \not\approx y \wedge \bigwedge_{P \in \sigma} P(x) \leftrightarrow P(y)$



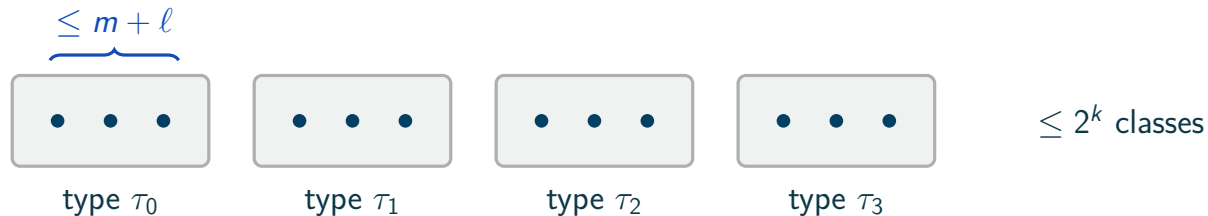
- A formula with ℓ quantifiers and m constants may enforce $\leq (m + \ell)$ representatives per 1-type.

$$\exists x_1 \exists x_2 \dots \exists x_\ell \bigwedge_{i=1}^{\ell} \bigwedge_{j=1}^m \bigwedge_{i' \neq i, j \neq j'} x_i \not\approx x_{i'} \wedge x_i \not\approx c_j \wedge c_j \not\approx c_{j'} \wedge \bigwedge_{P \in \sigma} P(x_i) \leftrightarrow P(x_j) \wedge P(x_i) \leftrightarrow P(c_j)$$

Lemma

Satisfiable φ with k predicates, m constants, ℓ quantifiers has a model with $\leq (m + \ell) \cdot 2^k$ elements.

Proof idea: keep up to $(m + \ell)$ representatives per class (or all, if fewer).



Why $m + \ell$ representatives per type suffice?

Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



- Constants are interpreted by points we keep from the start.

Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



- Constants are interpreted by points we keep from the start.
- During an ℓ -quantifier evaluation, at most ℓ variable values must remain distinct.

Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



- Constants are interpreted by points we keep from the start.
- During an ℓ -quantifier evaluation, at most ℓ variable values must remain distinct.
- If a fresh witness is needed, choose an unused kept point of the same type.

Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



- Constants are interpreted by points we keep from the start.
- During an ℓ -quantifier evaluation, at most ℓ variable values must remain distinct.
- If a fresh witness is needed, choose an unused kept point of the same type.

Conclusion: The formula cannot exhaust a reservoir of size $m + \ell$.

Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



- Constants are interpreted by points we keep from the start.
- During an ℓ -quantifier evaluation, at most ℓ variable values must remain distinct.
- If a fresh witness is needed, choose an unused kept point of the same type.

Conclusion: The formula cannot exhaust a reservoir of size $m + \ell$.

This implies:

Why $m + \ell$ representatives per type suffice?

Intuition: Think of constants and currently assigned variables as labelled **pebbles occupying** one 1-type class.



- Constants are interpreted by points we keep from the start.
- During an ℓ -quantifier evaluation, at most ℓ variable values must remain distinct.
- If a fresh witness is needed, choose an unused kept point of the same type.

Conclusion: The formula cannot exhaust a reservoir of size $m + \ell$.

This implies:

Theorem

Every satisfiable MFO sentence φ has a model of size $\leq |\varphi|2^{|\varphi|}$. SAT(MFO) is in NEXPTIME.

Towards the lower bound: Tilings

Towards the lower bound: Tilings

A finite set of *tile types* with coloured edges; a *tiling* makes touching edges match.

tiles



allowed: edges agree



forbidden: mismatch



Towards the lower bound: Tilings

A finite set of *tile types* with coloured edges; a *tiling* makes touching edges match.

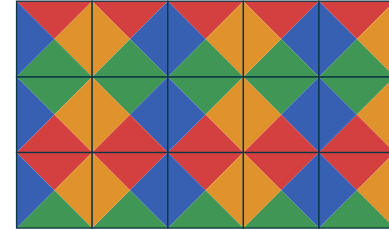
tiles



allowed: edges agree



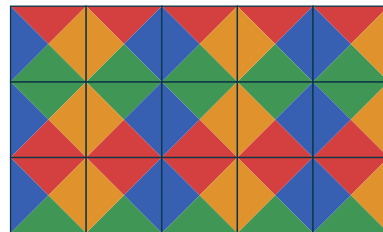
forbidden: mismatch



A correctly tiled rectangle — every pair of touching edges agrees.

Towards the lower bound: Tilings

A finite set of *tile types* with coloured edges; a *tiling* makes touching edges match.



A correctly tiled rectangle — every pair of touching edges agrees.

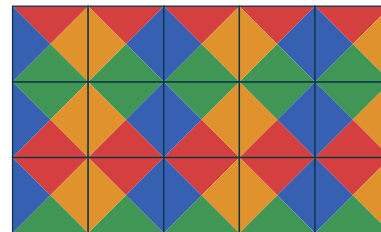
Definition (NEXPTIME-complete $2^n \times 2^n$ torus tiling)

Input: Tile types T with relations H, V and n in unary, and a tile $t_0 \in T$.

Output: Can the $2^n \times 2^n$ grid (edges wrap) be tiled, with t_0 at the origin?

Towards the lower bound: Tilings

A finite set of *tile types* with coloured edges; a *tiling* makes touching edges match.

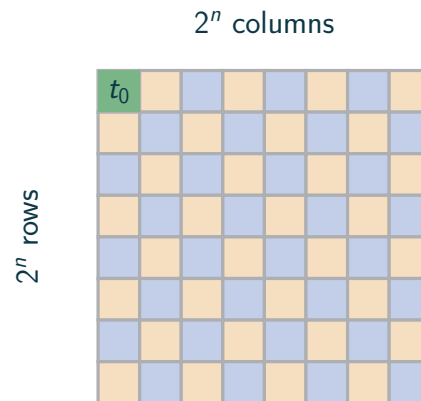


A correctly tiled rectangle — every pair of touching edges agrees.

Definition (NEXPTIME-complete $2^n \times 2^n$ torus tiling)

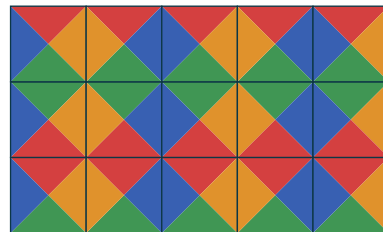
Input: Tile types T with relations H, V and n in unary, and a tile $t_0 \in T$.

Output: Can the $2^n \times 2^n$ grid (edges wrap) be tiled, with t_0 at the origin?



Towards the lower bound: Tilings

A finite set of *tile types* with coloured edges; a *tiling* makes touching edges match.



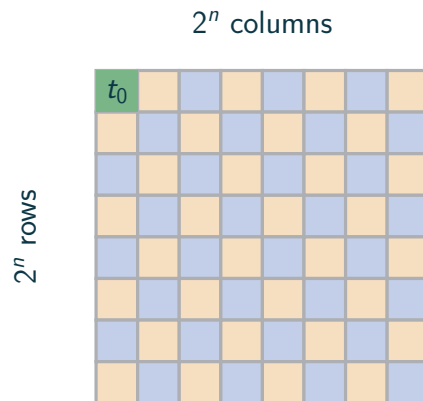
A correctly tiled rectangle — every pair of touching edges agrees.

Definition (NEXPTIME-complete $2^n \times 2^n$ torus tiling)

Input: Tile types T with relations H, V and n in unary, and a tile $t_0 \in T$.

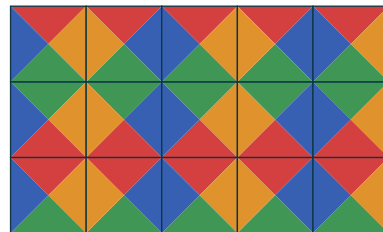
Output: Can the $2^n \times 2^n$ grid (edges wrap) be tiled, with t_0 at the origin?

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.



Towards the lower bound: Tilings

A finite set of *tile types* with coloured edges; a *tiling* makes touching edges match.



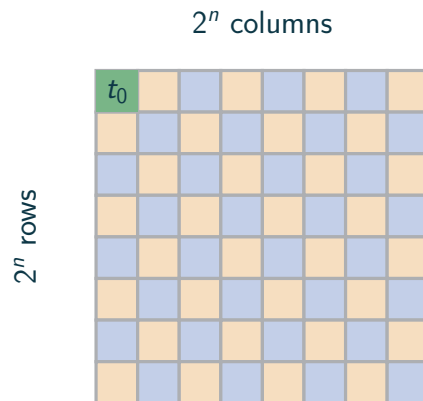
A correctly tiled rectangle — every pair of touching edges agrees.

Definition (NEXPTIME-complete $2^n \times 2^n$ torus tiling)

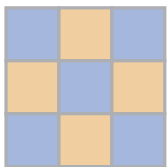
Input: Tile types T with relations H, V and n in unary, and a tile $t_0 \in T$.

Output: Can the $2^n \times 2^n$ grid (edges wrap) be tiled, with t_0 at the origin?

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

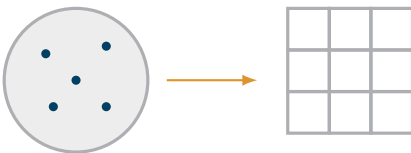


tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



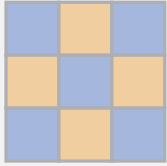
each coordinate realised;
pick a rep, read its tile.

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling

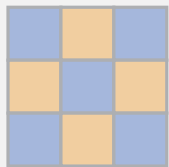


each coordinate realised;
pick a rep, read its tile.

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



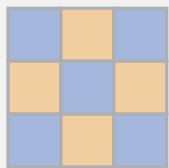
each coordinate realised;
pick a rep, read its tile.

1. Addresses — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



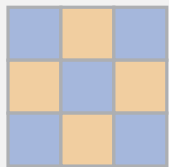
each coordinate realised;
pick a rep, read its tile.

- 1. Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
- 2. Successor** — a definable “+1” on coordinates (compare bits).

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



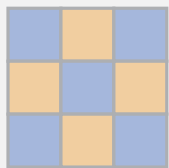
each coordinate realised;
pick a rep, read its tile.

1. **Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
2. **Successor** — a definable “+1” on coordinates (compare bits).
3. **Neighbours** — right / up neighbour = same row / column + successor.

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



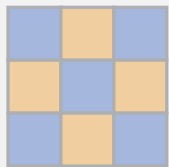
each coordinate realised;
pick a rep, read its tile.

1. **Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
2. **Successor** — a definable “+1” on coordinates (compare bits).
3. **Neighbours** — right / up neighbour = same row / column + successor.
4. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



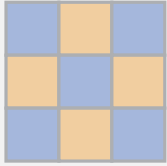
each coordinate realised;
pick a rep, read its tile.

1. **Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
2. **Successor** — a definable “+1” on coordinates (compare bits).
3. **Neighbours** — right / up neighbour = same row / column + successor.
4. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .
5. **Totality** — every cell has a right and an up neighbour (closure).

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



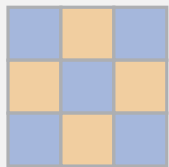
each coordinate realised;
pick a rep, read its tile.

1. **Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
2. **Successor** — a definable “+1” on coordinates (compare bits).
3. **Neighbours** — right / up neighbour = same row / column + successor.
4. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .
5. **Totality** — every cell has a right and an up neighbour (closure).
6. **One tile** — exactly one tile predicate per element.

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



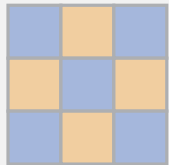
each coordinate realised;
pick a rep, read its tile.

1. **Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
2. **Successor** — a definable “+1” on coordinates (compare bits).
3. **Neighbours** — right / up neighbour = same row / column + successor.
4. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .
5. **Totality** — every cell has a right and an up neighbour (closure).
6. **One tile** — exactly one tile predicate per element.
7. **Compatibility** — matching tiles across every right (H) and up (V) step.

A reduction checklist: $\varphi_n = \text{grid} + \text{tiling}$

Goal: poly-size φ_n over $\{X_i, Y_i, T_t \mid t \in T, i \leq n\}$ with $\varphi_n \text{ sat} \Leftrightarrow \text{grid tileable}$.

tiling $\Rightarrow$ model



domain = cells; T_t from the tiling. All clauses hold.

model $\Rightarrow$ tiling



each coordinate realised;
pick a rep, read its tile.

1. **Addresses** — each element has an x - and y -coordinate in $\{0, 1\}^n$ (its bits).
2. **Successor** — a definable “+1” on coordinates (compare bits).
3. **Neighbours** — right / up neighbour = same row / column + successor.
4. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .
5. **Totality** — every cell has a right and an up neighbour (closure).
6. **One tile** — exactly one tile predicate per element.
7. **Compatibility** — matching tiles across every right (H) and up (V) step.
8. **Consistency** — equal coordinates $\Rightarrow$ equal tile.

How do we encode positions and their successors?

How do we encode positions and their successors?

- We employ the following monadic predicates to encode a position (x, y) carrying a tile t :

$$\underbrace{X_1, \dots, X_n}_{\text{column } x} \quad \underbrace{Y_1, \dots, Y_n}_{\text{row } y} \quad \underbrace{T_t \ (t \in T)}_{\text{tile}}$$

How do we encode positions and their successors?

- We employ the following monadic predicates to encode a position (x, y) carrying a tile t :

$$\underbrace{X_1, \dots, X_n}_{\text{column } x} \quad \underbrace{Y_1, \dots, Y_n}_{\text{row } y} \quad \underbrace{T_t \ (t \in T)}_{\text{tile}}$$

- **Idea:** X_i and Y_i encode the **binary representation** of the x - and y -**coordinate of a cell**, respectively.

How do we encode positions and their successors?

- We employ the following monadic predicates to encode a position (x, y) carrying a tile t :

$$\underbrace{X_1, \dots, X_n}_{\text{column } x} \quad \underbrace{Y_1, \dots, Y_n}_{\text{row } y} \quad \underbrace{T_t \ (t \in T)}_{\text{tile}}$$

- **Idea:** X_i and Y_i encode the **binary representation** of the x - and y -**coordinate of a cell**, respectively.
- **Formally:** If $\mathfrak{A} \models \varphi_n$ then $\mathfrak{A} \models X_i[a]$ iff the i -**th bit** of an x -coordinate of a cell represented by x **is on**.

How do we encode positions and their successors?

- We employ the following monadic predicates to encode a position (x, y) carrying a tile t :

$$\underbrace{X_1, \dots, X_n}_{\text{column } x} \quad \underbrace{Y_1, \dots, Y_n}_{\text{row } y} \quad \underbrace{T_t \ (t \in T)}_{\text{tile}}$$

- **Idea:** X_i and Y_i encode the **binary representation** of the x - and y -**coordinate of a cell**, respectively.
- **Formally:** If $\mathfrak{A} \models \varphi_n$ then $\mathfrak{A} \models X_i[a]$ iff the i -**th bit** of an x -coordinate of a cell represented by x **is on**.

Example: Suppose $\mathfrak{A} \models \varphi_3$, and let the cell $(5, 3)$ —encoded by the element $a_{5,3}$ —carry the tile t .

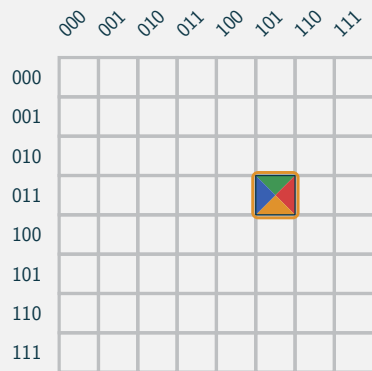
How do we encode positions and their successors?

- We employ the following monadic predicates to encode a position (x, y) carrying a tile t :

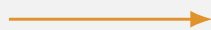
$$\underbrace{X_1, \dots, X_n}_{\text{column } x} \quad \underbrace{Y_1, \dots, Y_n}_{\text{row } y} \quad \underbrace{T_t \ (t \in T)}_{\text{tile}}$$

- Idea:** X_i and Y_i encode the **binary representation** of the x - and y -**coordinate of a cell**, respectively.
- Formally:** If $\mathfrak{A} \models \varphi_n$ then $\mathfrak{A} \models X_i[a]$ iff the i -th **bit** of an x -coordinate of a cell represented by x **is on**.

Example: Suppose $\mathfrak{A} \models \varphi_3$, and let the cell $(5, 3)$ —encoded by the element $a_{5,3}$ —carry the tile t .



8×8 address space



the tile predicate T_t

1-type of the element $a_{5,3}$

$$X_3 X_2 X_1(a_{5,3}) = 101$$

$$Y_3 Y_2 Y_1(a_{5,3}) = 011$$

$$T_t(a_{5,3}) \quad + \text{ all other } T_{t'} \text{ are false}$$

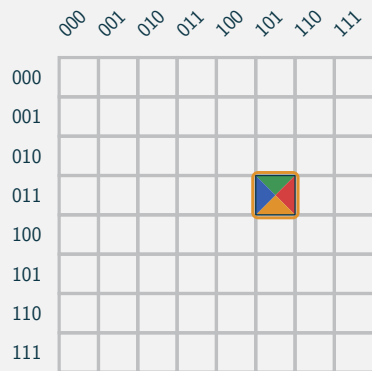
How do we encode positions and their successors?

- We employ the following monadic predicates to encode a position (x, y) carrying a tile t :

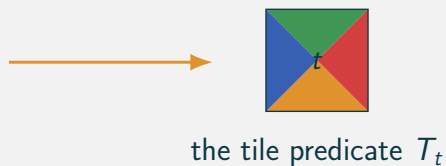
$$\underbrace{X_1, \dots, X_n}_{\text{column } x} \quad \underbrace{Y_1, \dots, Y_n}_{\text{row } y} \quad \underbrace{T_t (t \in T)}_{\text{tile}}$$

- Idea:** X_i and Y_i encode the **binary representation** of the x - and y -**coordinate of a cell**, respectively.
- Formally:** If $\mathfrak{A} \models \varphi_n$ then $\mathfrak{A} \models X_i[a]$ iff the i -th **bit** of an x -coordinate of a cell represented by x **is on**.

Example: Suppose $\mathfrak{A} \models \varphi_3$, and let the cell $(5, 3)$ —encoded by the element $a_{5,3}$ —carry the tile t .



8×8 address space



1-type of the element $a_{5,3}$

$$X_3 X_2 X_1(a_{5,3}) = 101$$

$$Y_3 Y_2 Y_1(a_{5,3}) = 011$$

$$T_t(a_{5,3}) \quad + \text{ all other } T_{t'} \text{ are false}$$

Then $a_{(5,3)}$ in $\mathfrak{A}$ satisfies: $X_3(x) \wedge \neg X_2(x) \wedge X_1(x) \wedge \neg Y_3(x) \wedge Y_2(x) \wedge Y_1(x) \wedge T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x)$

Neighbours: Up and Down Macros & “+1” gadget

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$

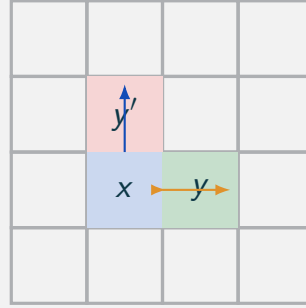
Up successor macro:

$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$



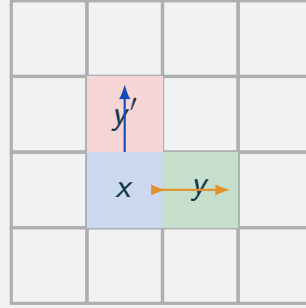
Up successor macro:

$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$



Up successor macro:

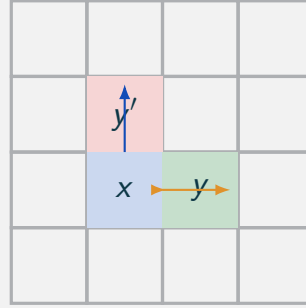
$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

How to define succ?

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$



Up successor macro:

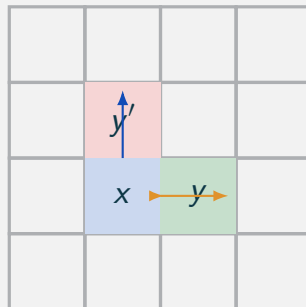
$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

How to define succ? Binary addition!

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$

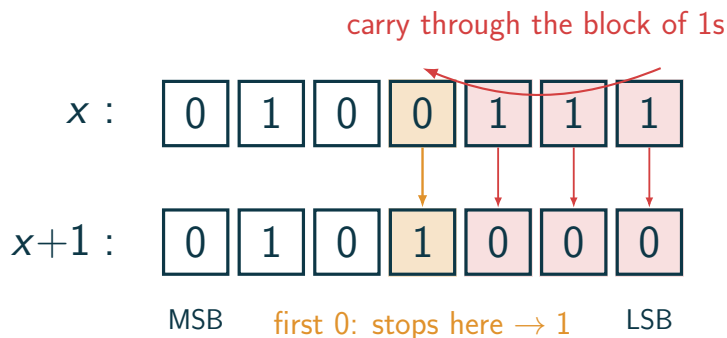


Up successor macro:

$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

How to define succ? Binary addition!

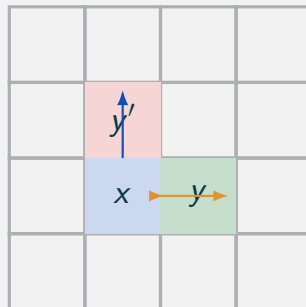
Main idea: From the least-significant bit: flip 1s to 0 (the carry), until the first 0, which becomes 1.



Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$



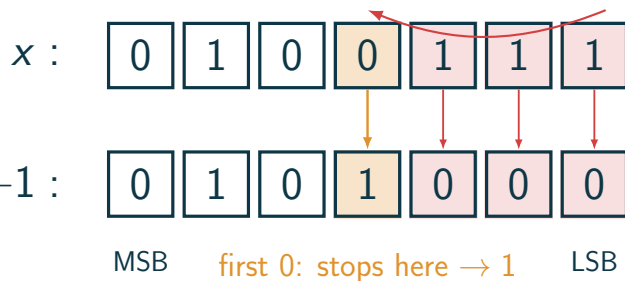
Up successor macro:

$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

How to define succ? Binary addition!

Main idea: From the least-significant bit: flip 1s to 0 (the carry), until the first 0, which becomes 1.

carry through the block of 1s

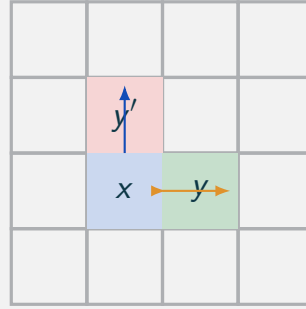


$$\text{SUCC}_X(x, y) := \text{WRAP}_X(x, y) \vee \bigvee_{j=1}^n \text{STEP}_j^X(x, y)$$

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$



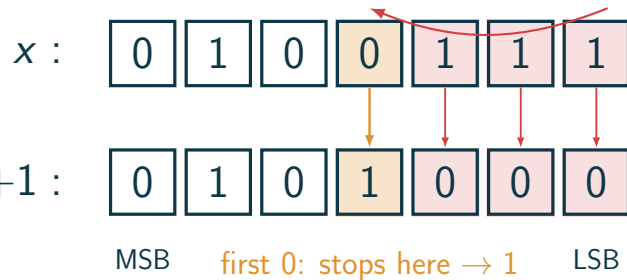
Up successor macro:

$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

How to define succ? Binary addition!

Main idea: From the least-significant bit: flip 1s to 0 (the carry), until the first 0, which becomes 1.

carry through the block of 1s



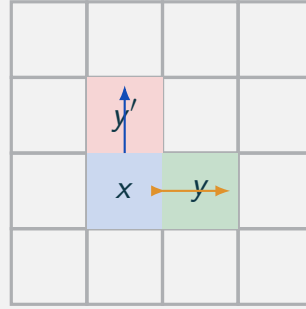
$$\text{SUCC}_X(x, y) := \text{WRAP}_X(x, y) \vee \bigvee_{j=1}^n \text{STEP}_j^X(x, y)$$

$$\text{STEP}_j^X(x, y) := \left(\bigwedge_{i < j} X_i(x) \wedge \neg X_i(y) \right) \wedge \neg X_j(x) \wedge X_j(y) \wedge \bigwedge_{i > j} (X_i(x) \leftrightarrow X_i(y))$$

Neighbours: Up and Down Macros & “+1” gadget

Right successor macro:

$$R(x, y) := \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \wedge \text{SUCC}_X(x, y)$$



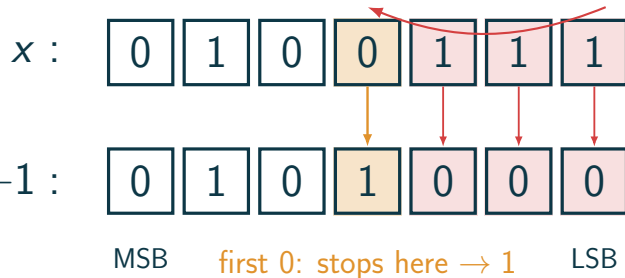
Up successor macro:

$$U(x, y) := \bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \text{SUCC}_Y(x, y)$$

How to define succ? Binary addition!

Main idea: From the least-significant bit: flip 1s to 0 (the carry), until the first 0, which becomes 1.

carry through the block of 1s



$$\text{SUCC}_X(x, y) := \text{WRAP}_X(x, y) \vee \bigvee_{j=1}^n \text{STEP}_j^X(x, y)$$

$$\text{STEP}_j^X(x, y) := \left(\bigwedge_{i < j} X_i(x) \wedge \neg X_i(y) \right) \wedge \neg X_j(x) \wedge X_j(y) \wedge \bigwedge_{i > j} (X_i(x) \leftrightarrow X_i(y))$$

$$\text{WRAP}_X(x, y) := \bigwedge_i X_i(x) \wedge \bigwedge_i \neg X_i(y)$$

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. Addresses + Successor + Neighbours



Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses** + **Successor** + **Neighbours**
2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses** + **Successor** + **Neighbours**

2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .

$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$



Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**

2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .

$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element.

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element.



$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element.



$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

5. **Compatibility** — matching tiles across every right (H) and up (V) step.

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element.



$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

5. **Compatibility** — matching tiles across every right (H) and up (V) step.



$$\forall x \forall y R(x, y) \rightarrow \bigvee_{(t, t') \in H} (T_t(x) \wedge T_{t'}(y))$$

$$\forall x \forall y U(x, y) \rightarrow \bigvee_{(t, t') \in V} (T_t(x) \wedge T_{t'}(y))$$

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element.



$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

5. **Compatibility** — matching tiles across every right (H) and up (V) step.



$$\forall x \forall y R(x, y) \rightarrow \bigvee_{(t, t') \in H} \left(T_t(x) \wedge T_{t'}(y) \right)$$

$$\forall x \forall y U(x, y) \rightarrow \bigvee_{(t, t') \in V} \left(T_t(x) \wedge T_{t'}(y) \right)$$

6. **Consistency** — equal coordinates $\Rightarrow$ equal tile.

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours**



2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 .



$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure).



$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element.



$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

5. **Compatibility** — matching tiles across every right (H) and up (V) step.



$$\forall x \forall y R(x, y) \rightarrow \bigvee_{(t, t') \in H} (T_t(x) \wedge T_{t'}(y))$$

$$\forall x \forall y U(x, y) \rightarrow \bigvee_{(t, t') \in V} (T_t(x) \wedge T_{t'}(y))$$

6. **Consistency** — equal coordinates $\Rightarrow$ equal tile. **Why needed?**

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours** ✓

2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 . ✓

$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure). ✓

$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element. ✓

$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

5. **Compatibility** — matching tiles across every right (H) and up (V) step. ✓

$$\forall x \forall y R(x, y) \rightarrow \bigvee_{(t, t') \in H} (T_t(x) \wedge T_{t'}(y))$$

$$\forall x \forall y U(x, y) \rightarrow \bigvee_{(t, t') \in V} (T_t(x) \wedge T_{t'}(y))$$

6. **Consistency** — equal coordinates $\Rightarrow$ equal tile. **Why needed?** ✓

$$\forall x \forall y \left(\bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \rightarrow \bigwedge_{t \in T} (T_t(x) \leftrightarrow T_t(y)) \right)$$

Filling-in our checklist: $\varphi_n = \text{grid} + \text{tiling}$

1. **Addresses + Successor + Neighbours** ✓

2. **Origin** — a start cell $(0, 0)$ carrying tile t_0 . ✓

$$\exists x \left(\bigwedge_i \neg X_i(x) \wedge \bigwedge_i \neg Y_i(x) \wedge T_{t_0}(x) \right)$$

3. **Totality** — every cell has a right and an up neighbour (closure). ✓

$$\forall x \exists y R(x, y) \wedge \forall x \exists y U(x, y)$$

4. **One tile** — exactly one tile predicate per element. ✓

$$\forall x \bigvee_{t \in T} \left(T_t(x) \wedge \bigwedge_{t' \neq t} \neg T_{t'}(x) \right)$$

5. **Compatibility** — matching tiles across every right (H) and up (V) step. ✓

$$\forall x \forall y R(x, y) \rightarrow \bigvee_{(t, t') \in H} (T_t(x) \wedge T_{t'}(y))$$

$$\forall x \forall y U(x, y) \rightarrow \bigvee_{(t, t') \in V} (T_t(x) \wedge T_{t'}(y))$$

6. **Consistency** — equal coordinates $\Rightarrow$ equal tile. **Why needed?** ✓

$$\forall x \forall y \left(\bigwedge_i (X_i(x) \leftrightarrow X_i(y)) \wedge \bigwedge_i (Y_i(x) \leftrightarrow Y_i(y)) \rightarrow \bigwedge_{t \in T} (T_t(x) \leftrightarrow T_t(y)) \right)$$

Theorem

The satisfiability problem for MFO is NEXPTIME-hard (even its two-variable pure fragment).

Take-away messages: MFO is NExpTime-complete.

Take-away messages: MFO is NExpTime-complete.

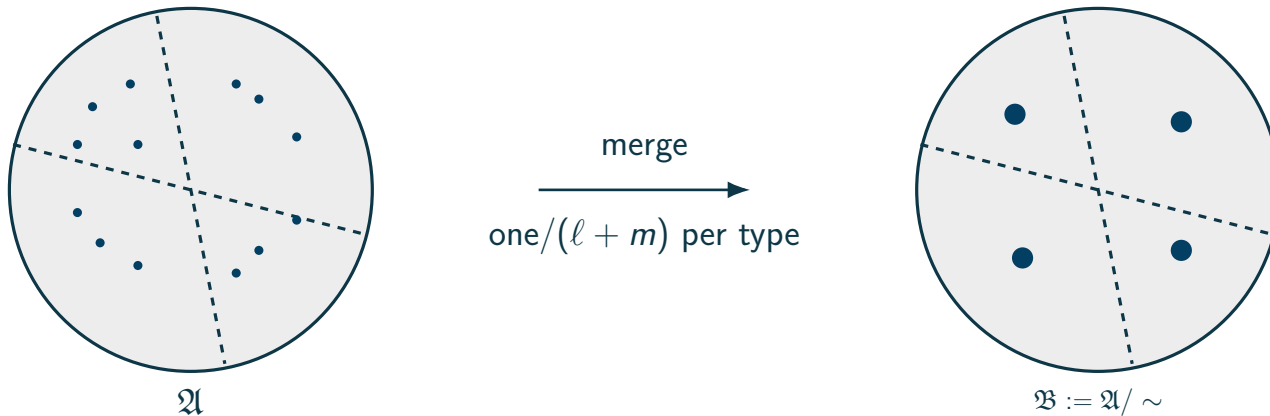
Theorem

Every satisfiable MFO sentence φ has a model of size $\leq |\varphi|2^{|\varphi|}$, so SAT(MFO) is in NEXPTIME.

Take-away messages: MFO is NExpTime-complete.

Theorem

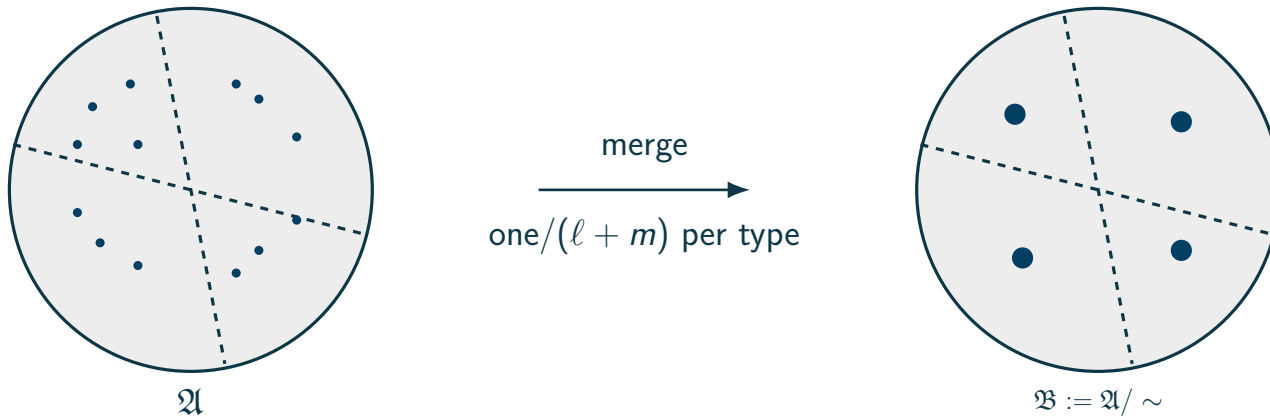
Every satisfiable MFO sentence φ has a model of size $\leq |\varphi|2^{|\varphi|}$, so $\text{SAT}(\text{MFO})$ is in NEXPTIME .



Take-away messages: MFO is NExpTime-complete.

Theorem

Every satisfiable MFO sentence φ has a model of size $\leq |\varphi|2^{|\varphi|}$, so $\text{SAT}(\text{MFO})$ is in NExpTime .



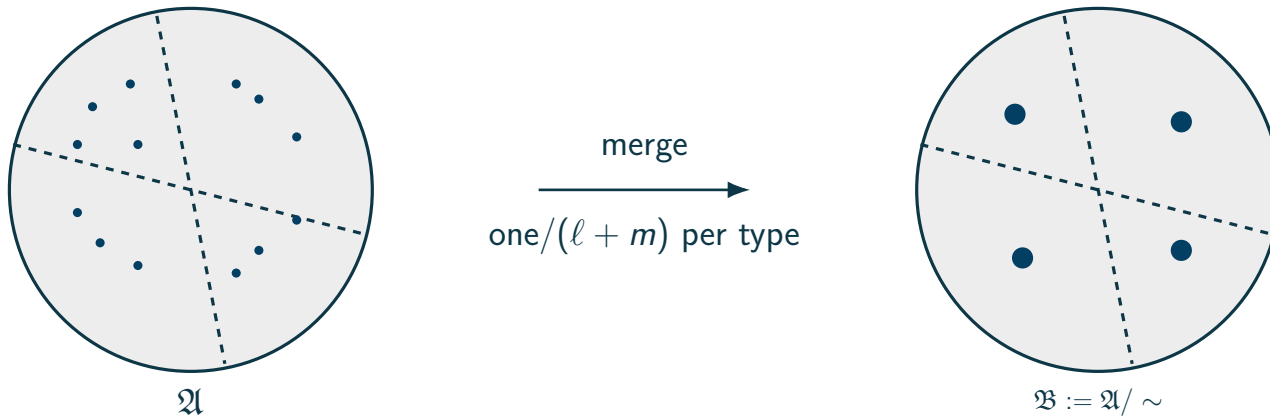
Theorem

$\text{SAT}(\text{MFO})$ is NExpTime -hard, even for its two-variable pure fragment, without $\approx$ and constants.

Take-away messages: MFO is NExpTime-complete.

Theorem

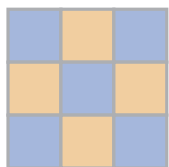
Every satisfiable MFO sentence φ has a model of size $\leq |\varphi|2^{|\varphi|}$, so $\text{SAT}(\text{MFO})$ is in NExpTime .



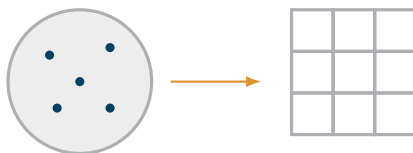
Theorem

$\text{SAT}(\text{MFO})$ is NExpTime -hard, even for its two-variable pure fragment, without $\approx$ and constants.

tiling $\Rightarrow$ model



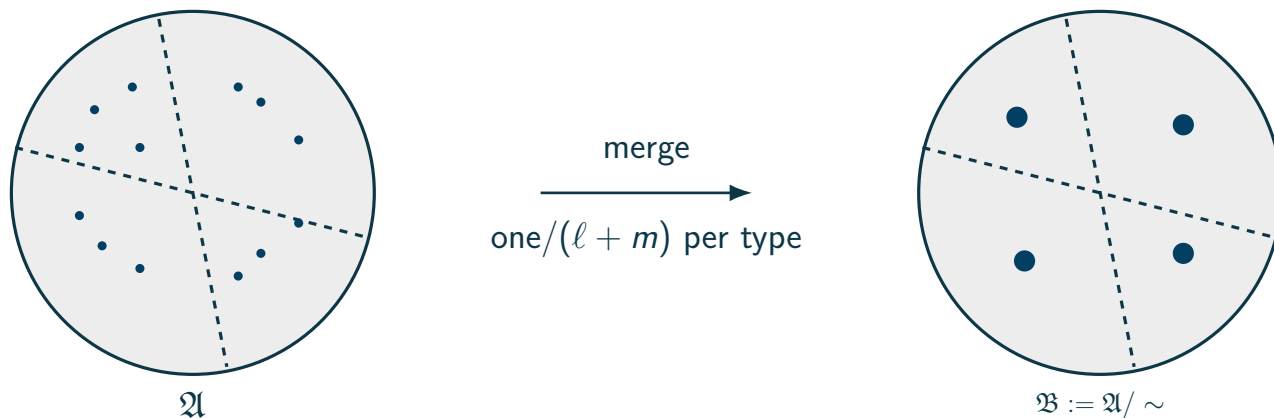
model $\Rightarrow$ tiling



Take-away messages: MFO is NExpTime-complete.

Theorem

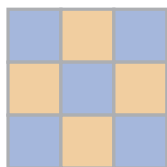
Every satisfiable MFO sentence φ has a model of size $\leq |\varphi|2^{|\varphi|}$, so $\text{SAT}(\text{MFO})$ is in NExpTime .



Theorem

$\text{SAT}(\text{MFO})$ is NExpTime -hard, even for its two-variable pure fragment, without $\approx$ and constants.

tiling $\Rightarrow$ model



model $\Rightarrow$ tiling

